Project Report

School of Arts and Sciences

Department of Computer Science and Mathematics

Submitted by:

Aya Al Saleh
202304825

Malak Khalil
202303748

Talia El Helou
202203451

Submitted to Dr. Louma Chaddad for the course "CSC430": Computer Networks

Submission Date: 26 April 2026

Abstract

This report presents the design and implementation of a multithreaded HTTP/HTTPS proxy server with advanced features including content caching, domain blacklisting, structured logging, and a web-based admin interface. The proxy forwards client requests to origin servers, caches responses for faster repeated access, blocks blacklisted domains, and provides real-time statistics via an intuitive dashboard. The project meets all requirements (A–I) and includes bonus features (HTTPS tunneling and admin interface). Each team member contributed distinct modules, which are detailed in separate sections. Comprehensive testing demonstrates the correctness and performance of the proxy.

Contents

1 Project Overview **5**

2 Overall Architecture and Requirements Matrix **6**

3 Malak's Contribution (Parts A, B, C) – Detailed **7**
3.1 Overview of Responsibilities . . . 7
3.2 Part A: Basic Proxy Functionality . . . 7
3.2.1 Design Decision . . . 7
3.2.2 Implementation Details . . . 7
3.2.3 Testing and Results . . . 8
3.3 Part B: Socket Programming . . . 10
3.3.1 Design Decision . . . 10
3.3.2 Implementation Details . . . 10
3.3.3 Testing and Results . . . 11
3.4 Part C: Request Parsing . . . 12
3.4.1 Design Decision . . . 12
3.4.2 Implementation Details . . . 12
3.4.3 Testing and Results . . . 12
3.5 Function-Level Summary of My Contribution . . . 13
3.6 Conclusion of Malak's Contribution . . . 14

4 Talia's Contribution (Parts E, F, H) – Detailed **15**
4.1 Overview of Responsibilities . . . 15
4.2 Part E: Structured Logging . . . 15
4.2.1 Design Decision . . . 15
4.2.2 Implementation Details . . . 15
4.2.3 Testing and Results . . . 16
4.3 Part F: Content Caching . . . 16
4.3.1 Design Decision . . . 16
4.3.2 Implementation Details . . . 17
4.3.3 Testing and Results . . . 17
4.4 Part H: HTTPS Proxy / MITM (Bonus) . . . 18
4.4.1 Design Decision . . . 18
4.4.2 Implementation Details . . . 19
4.4.3 Testing and Results . . . 19
4.5 Conclusion of Talia's Contribution . . . 20

5 Aya's Contribution (Parts D, G, I) – Detailed **21**
5.1 Overview of Responsibilities . . . 21
5.2 Multithreading & High-Concurrency Handling (Part D) . . . 21
5.2.1 Design Decision . . . 21
5.2.2 Implementation Details . . . 21
5.2.3 Testing and Results . . . 22
5.3 Blacklist / Whitelist (Part G) . . . 22
5.3.1 Design Decision . . . 22
5.3.2 Implementation Details . . . 22
5.3.3 Testing and Results . . . 22

5.4 Admin Interface (Part I – Bonus) 23
5.4.1 Design Decision 23
5.4.2 Implementation Details 23
5.4.3 Testing and Results 24
5.5 Conclusion of Aya's Contribution 25

6 Testing Results Summary **26**

7 Conclusion **27**

8 Appendix **27**
8.1 Submitted Project Files 27
8.2 Representative Test Commands 28
8.3 Notes on Testing 28
8.4 Final Submission Note 28

1 Project Overview

The proxy server listens on `127.0.0.1:8888` and forwards HTTP and HTTPS (via CONNECT) requests from clients to origin servers. It supports:

- **Basic forwarding** (HTTP/HTTPS)
- **Multithreading** – concurrent handling of multiple clients
- **Content caching** – in-memory cache with expiration based on `Cache-Control`, `Expires`, or a default TTL
- **Blacklisting** – block specific domains with a 403 Forbidden response
- **Structured logging** – JSONL format logs of requests, responses, cache events, and errors
- **Admin interface** – a web dashboard (virtual domain `proxy.admin`) for monitoring stats, viewing logs, and managing the blacklist

The proxy is implemented in pure Python using only the standard library (no external dependencies).

2 Overall Architecture and Requirements Matrix

The following table summarizes which team member implemented each requirement:

ID	Requirement	Team Member
A	Basic HTTP forwarding	Malak
B	Socket programming	Malak
C	Request parsing	Malak
D	Multithreading	Aya
E	Structured logging	Talia
F	Content caching	Talia
G	Blacklist/Whitelist	Aya
H	HTTPS proxy (bonus)	Talia
I	Admin interface (bonus)	Aya

Table 1: Distribution of responsibilities

All features have been implemented and tested.

3 Malak's Contribution (Parts A, B, C) – Detailed

3.1 Overview of Responsibilities

My contribution covered the core proxy logic required for Parts A, B, and C of the project. In particular, I implemented the main HTTP forwarding path, the low-level socket communication between the client and the origin server, and the HTTP parsing logic needed to understand and rebuild requests and responses. I also divided this functionality into helper functions so that the final code would be easier to test, debug, and integrate with the other modules. In the integrated version, my code forms the normal HTTP processing path, while Aya's blacklist/admin logic and Talia's logging, caching, and HTTPS logic are layered around it.

3.2 Part A: Basic Proxy Functionality

3.2.1 Design Decision

For Part A, I chose to implement the proxy workflow as a clear sequence of steps. The proxy first receives a request from the client, then identifies the destination host, port, and path, forwards the request to the origin server, reads the origin server's response, and finally relays that response back to the client. I kept this path modular so that normal HTTP requests could be handled by my code, while HTTPS requests could be recognized and sent to the secure HTTPS module.

Another design decision was to support persistent HTTP connections. Instead of forcing one request per connection, I added helper logic to determine whether the client and the origin server want to keep the connection alive. This makes the proxy closer to real HTTP/1.1 behavior.

3.2.2 Implementation Details

The main HTTP path is implemented through `handle_client()` and `handle_http_request()`. The function `handle_client()` reads requests in a loop, parses the request line, and decides whether the request is for the admin interface, an HTTPS `CONNECT` request, or a normal HTTP request. For ordinary HTTP traffic, it calls `handle_http_request()`.

Inside `handle_http_request()`, the proxy first calls `get_destination_for_http()` to determine the target host, port, and path. Then it checks whether the client requested a persistent connection by calling `should_keep_alive_request()`. After that, the proxy either reuses an already-open socket for the same origin server or opens a new connection. The request is rebuilt with `build_forward_request()` and sent to the origin server using `sendall()`.

Once the request has been forwarded, the proxy reads the complete response using `read_http_response()`. This function parses the response status line, headers, and body, and also determines whether the origin connection can remain open. The proxy then rebuilds the response for the client using `build_client_response()`, especially to ensure the correct `Content-Length` and `Connection` headers. If the response comes from cache, the helper `adapt_cached_response_for_client()` updates the cached response so that its connection behavior matches the current client request.

In addition to normal HTTP forwarding, my code also contributes to the HTTPS path at the proxy entry point. In `handle_client()`, the proxy reads and parses the client re-

quest, checks whether the method is `CONNECT`, validates the destination host and port using `parse_host_port()`, and then dispatches the connection to the HTTPS handler. This means that my part is responsible for correctly recognizing HTTPS proxy requests and routing them to the proper secure-processing module. At the same time, the normal HTTP forwarding path explicitly rejects `https://...` URLs in `get_destination_for_http()`, because HTTPS must be handled through `CONNECT` rather than the ordinary HTTP forwarding logic. The actual HTTPS interception, certificate generation, and TLS processing are implemented in Talia's module, but my code is the part that detects the HTTPS request and sends it to the correct handler.

I also implemented `send_error_response()` so the proxy can generate valid HTTP error messages such as `400 Bad Request`, `403 Forbidden`, `500 Internal Server Error`, and `502 Bad Gateway`. This makes the proxy behave like a real network application instead of simply closing the connection when an error occurs.

3.2.3 Testing and Results

I tested the basic forwarding path with a local test server created using:

Listing 1: Starting a local origin server

```
python -m http.server 8000
```

Figure 1: Local origin server running on port 8000. This screenshot shows the test server started successfully using Python's built-in `http.server` module, which was used as the destination server for validating the proxy forwarding path.

Then I sent a request through the proxy using:

Listing 2: HTTP forwarding test

```
curl.exe -x http://127.0.0.1:8888 http://127.0.0.1:8000 -v
```

The expected result was a successful `200 OK` response containing the contents of `index.html`. In our testing, the response body returned:

Listing 3: Expected forwarded response body

```
<p>Welcome to the Caching Proxy Server!</p>
```

This confirmed that the proxy successfully accepted the client request, forwarded it to the origin server, received the response, and relayed it back to the client.

```
PS C:\Users\USP\OneDrive\Documents\CachingProxyServer> curl.exe -x http://127.0.0.1:8888 http://127.0.0.1:8000 -v
*   Trying 127.0.0.1:8888...
* Established connection to 127.0.0.1 (127.0.0.1 port 8888) from 127.0.0.1 port 55797
* using HTTP/1.x
> GET http://127.0.0.1:8000/ HTTP/1.1
> Host: 127.0.0.1:8000
> User-Agent: curl/8.18.0
> Accept: */*
> Proxy-Connection: Keep-Alive
>
* Request completely sent off
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< server: SimpleHTTP/0.6 Python/3.13.3
< date: Sat, 25 Apr 2026 21:45:50 GMT
< content-type: text/html
< content-length: 43
< last-modified: Sat, 25 Apr 2026 18:24:22 GMT
* HTTP/1.0 connection set to keep alive
< connection: keep-alive
<
<p>Welcome to the Caching Proxy Server!</p>* Connection #0 to host 127.0.0.1:8888 left intact
```

Figure 2: Successful HTTP forwarding through the proxy. The screenshot shows the client sending the request to `127.0.0.1:8888`, the proxy forwarding it to the local server on port 8000, and the returned `200 OK` response with the expected HTML content.

I also tested forwarding of a request with a body using:

Listing 4: POST forwarding test

```
curl.exe -x http://127.0.0.1:8888 -d "hello=world" http
   ://127.0.0.1:8000 -v
```

The local Python test server returned `501 Unsupported method ('POST')`. However, this was expected, because `python -m http.server` does not implement POST. The important result is that the proxy itself did not fail, which proves that it successfully read and forwarded the request body.

I also tested the HTTPS path through the proxy using:

Listing 5: HTTPS request test through the proxy

```
curl.exe -k -x http://127.0.0.1:8888 https://google.com -v
```

The expected result was first:

Listing 6: Expected CONNECT success message

```
HTTP/1.1 200 Connection Established
```

followed by a valid HTTPS server response, which in our testing included a redirect such as `301 Moved Permanently`. This test proved that the proxy correctly recognized the `CONNECT` request, extracted the target host and port, and dispatched the HTTPS connection to the secure handling module. Therefore, even though the TLS interception itself belongs to the HTTPS bonus part, my core proxy logic still plays an essential role in enabling HTTPS support.

```
(venv) PS C:\Users\USP\OneDrive\Documents\CachingProxyServer> curl.exe -k -x http://127.0.0.1:8888 https://google.com -v
*   Trying 127.0.0.1:8888...
* CONNECT: no ALPN negotiated
* allocate connect buffer
* Establish HTTP proxy tunnel to google.com:443
> CONNECT google.com:443 HTTP/1.1
> Host: google.com:443
> User-Agent: curl/8.18.0
> Proxy-Connection: Keep-Alive
>
< HTTP/1.1 200 Connection Established
<
* CONNECT phase completed
* CONNECT tunnel established, response 200
* schannel: disabled automatic use of client certificate
* ALPN: curl offers http/1.1
* ALPN: server did not agree on a protocol. Uses default.
* Established connection to 127.0.0.1 (127.0.0.1 port 8888) from 127.0.0.1 port 55662
* using HTTP/1.x
> GET / HTTP/1.1
> Host: google.com
> User-Agent: curl/8.18.0
> Accept: */*
>
* Request completely sent off
* schannel: remote party requests renegotiation
* schannel: renegotiating SSL/TLS connection
* schannel: SSL/TLS connection renegotiated
* schannel: remote party requests renegotiation
* schannel: renegotiating SSL/TLS connection
* schannel: SSL/TLS connection renegotiated
< HTTP/1.1 301 Moved Permanently
< Location: https://www.google.com/
< Content-Type: text/html; charset=UTF-8
< Content-Security-Policy-Report-Only: object-src 'none';base-uri 'self';script-src 'nonce-AmcBkKblJEvt-t3s6T_wJw' 'strict-dynamic' '
```

Figure 3: HTTPS request recognition through the proxy. The screenshot shows a successful `CONNECT` request and the `200 Connection Established` response, proving that the proxy correctly recognizes HTTPS proxy traffic, extracts the destination host and port, and dispatches the request to the secure HTTPS handling module.

3.3 Part B: Socket Programming

3.3.1 Design Decision

For Part B, I used low-level TCP sockets directly, since this matches the socket programming concepts covered in the course. The proxy manages two communication sides: one socket between the client and the proxy, and another socket between the proxy and the origin server. I also used socket timeouts and helper functions to make socket reading more reliable and easier to debug.

A second design decision was to separate different reading behaviors into different helper functions, because HTTP messages can end in different ways. Some bodies use `Content-Length`, some use chunked transfer encoding, and some end only when the remote side closes the connection.

3.3.2 Implementation Details

The server-side socket setup is initialized in `start_proxy()`, where the proxy creates a TCP socket, enables `SO_REUSEADDR`, binds to `127.0.0.1:8888`, and starts listening for incoming connections.

For communication, I implemented the following helpers:

- `recv_until_header_end()` reads from a socket until the end of the HTTP headers is reached.

- `read_exact_bytes()` reads an exact number of bytes when the body length is known from `Content-Length`.
- `read_until_socket_close()` reads data until the peer closes the connection.
- `close_origin_socket()` safely closes and removes an origin socket from the saved connection pool.

When forwarding a request, `handle_http_request()` creates or reuses a remote socket using:

Listing 7: Connecting to the origin server

```
remote_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
remote_socket.settimeout(SOCKET_TIMEOUT)
remote_socket.connect((host, port))
```

The request is then sent with `sendall()`, and the response is read back using the helper functions above.

3.3.3 Testing and Results

I tested the socket layer through the complete proxy workflow. First, I started the proxy using:

Listing 8: Starting the proxy server

```
python proxy_server.py
```

The proxy printed:

Listing 9: Expected startup message

```
[*] LAU Proxy LIVE on 127.0.0.1:8888
```

This screenshot confirms that the proxy successfully initialized its listening socket and was ready to accept client connections on the configured port.

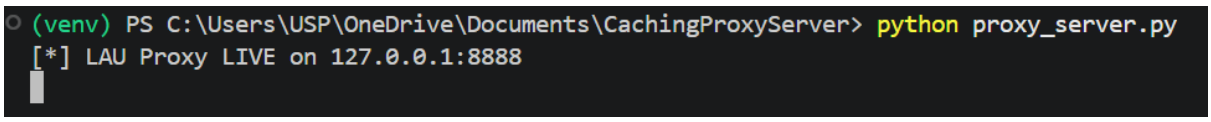

Figure 4: Proxy server startup on `127.0.0.1:8888`. This confirms that the proxy successfully created a listening TCP socket, bound it to the designated local address and port, and became ready to accept incoming client connections.

Then I started the local server on port 8000 and ran:

Listing 10: Socket communication test through the proxy

```
curl.exe -x http://127.0.0.1:8888 http://127.0.0.1:8000 -v
```

Since the proxy successfully accepted a client connection, opened a second connection to the origin server, and relayed the response correctly, this demonstrated that the socket programming part was functioning as intended.

3.4 Part C: Request Parsing

3.4.1 Design Decision

For Part C, I wanted the parser to follow the real structure of HTTP messages instead of treating the request as one raw block of text. Because of this, I divided parsing into several stages: parsing the request line, collecting headers into a dictionary, deciding how to read the body, extracting the destination host and port, and rebuilding the request before forwarding it.

I also chose to support both of the main body formats used in HTTP: `Content-Length` and `Transfer-Encoding: chunked`. Even if chunked requests are less common in our classroom testing, supporting them makes the proxy more correct and realistic.

3.4.2 Implementation Details

The function `read_http_request()` reads one full request from the client socket. It separates the headers from the body, decodes the headers using `iso-8859-1`, and stores them in a dictionary with lowercase header names. If the request uses chunked transfer encoding, it calls `read_chunked_body()`; otherwise, it reads the body according to `Content-Length`.

The request line itself is parsed by `parse_request_line()`, which returns `(method, target, version)`. The target destination is resolved by `get_destination_for_http()`, which supports:

- absolute-form requests such as `GET http://example.com/index.html HTTP/1.1`
- origin-form requests such as `GET /index.html HTTP/1.1`, where the destination host comes from the `Host` header

The helper `parse_host_port()` extracts the host and port and validates the port number. On the response side, I also implemented `read_http_response()`, `response_has_body()`, and the keep-alive helper functions so the proxy can fully understand the server's reply.

Finally, `build_forward_request()` removes headers that should not be forwarded as-is, such as `Proxy-Connection`, `Keep-Alive`, `TE`, and `Upgrade`, then rebuilds a clean request with the correct `Host`, `Content-Length`, and `Connection` headers.

3.4.3 Testing and Results

I tested parsing using both a simple GET request and a POST request with a body. The GET test was:

Listing 11: GET parsing test

```
curl.exe -x http://127.0.0.1:8888 http://127.0.0.1:8000 -v
```

This test proved that the proxy correctly parsed the request line, the absolute URL, the `Host` header, and the HTTP version.

The POST test was:

Listing 12: Body parsing test

```
curl.exe -x http://127.0.0.1:8888 -d "hello=world" http
   ://127.0.0.1:8000 -v
```

```
(venv) PS C:\Users\USP\OneDrive\Documents\CachingProxyServer> curl.exe -x http://127.0.0.1:8888 -d "hello=world" http://127.0.0.1:800
0 -v
> Content-Length: 11
> Content-Type: application/x-www-form-urlencoded
>
* upload completely sent off: 11 bytes
* HTTP 1.0, assume close after body
< HTTP/1.0 501 Unsupported method ('POST')
< server: SimpleHTTP/0.6 Python/3.13.3
< date: Sat, 25 Apr 2026 21:48:54 GMT
< content-type: text/html;charset=utf-8
< content-length: 357
* HTTP/1.0 connection set to keep alive
< connection: keep-alive
<
<!DOCTYPE HTML>
<html lang="en">
    <head>
        <meta charset="utf-8">
        <title>Error response</title>
    </head>
    <body>
        <h1>Error response</h1>
        <p>Error code: 501</p>
        <p>Message: Unsupported method ('POST').</p>
        <p>Error code explanation: 501 - Server does not support this operation.</p>
    </body>
</html>
* Connection #0 to host 127.0.0.1:8888 left intact
```

Figure 5: POST request and body parsing test through the proxy. The screenshot shows that the proxy successfully parsed and forwarded a request body using `Content-Length`. The `501 Unsupported method ('POST')` response was generated by the local Python test server, not by the proxy itself.

This proved that the proxy correctly parsed a request body using `Content-Length` and forwarded it, even though the origin server later responded with `501 Unsupported method ('POST')`.

In addition, the proxy correctly rebuilt and returned the server's HTTP response, which showed that both request-side and response-side parsing were working correctly.

3.5 Function-Level Summary of My Contribution

For clarity, the main functions I implemented for Parts A, B, and C are grouped as follows:

- **Error handling and socket I/O helpers:** `send_error_response()`, `recv_until_header_end()` `read_exact_bytes()`, `read_until_socket_close()`, and `close_origin_socket()`.
- **HTTP body and request parsing helpers:** `read_chunked_body()`, `read_http_request()`, `parse_request_line()`, `parse_host_port()`, and `get_destination_for_http()`.
- **Persistent connection and forwarding helpers:** `should_keep_alive_request()`, `should_keep_alive_response()`, and `build_forward_request()`.
- **Response parsing and rebuilding helpers:** `response_has_body()`, `read_http_response()`, `build_client_response()`, and `adapt_cached_response_for_client()`.
- **Main integrated processing functions:** `handle_http_request()` and `handle_client()`.

These functions together implement the core proxy workflow: receiving a client request, parsing it, identifying the destination server, forwarding the request, receiving the response, rebuilding the final response, and sending it back to the client.

3.6 Conclusion of Malak's Contribution

I successfully implemented Parts A with the bonus, B, and C of the project by building the core HTTP proxy path, the socket communication layer, and the request/response parsing logic. My code handles request reception, destination extraction, header parsing, body handling, request rebuilding, response parsing, error responses, persistent HTTP behavior, and HTTPS request recognition at the proxy entry point. These functions form the foundation of the final integrated proxy and were later combined with Aya's multithreading/blacklist/admin modules and Talia's logging/cache/HTTPS modules.

4 Talia's Contribution (Parts E, F, H) – Detailed

4.1 Overview of Responsibilities

Talia's contribution covered Parts E, F, and H of the project. More specifically, she implemented the structured logging system, the in-memory response cache, and the HTTPS proxy support. In the final integrated version, these modules are imported into the main proxy server and are used together with the core HTTP forwarding logic. The logging module records all important proxy events, the cache module stores reusable HTTP responses, and the HTTPS module enables secure interception of HTTPS traffic using a local certificate authority.

4.2 Part E: Structured Logging

4.2.1 Design Decision

For the logging component, the main design goal was to make the proxy's behavior easy to monitor and debug. Instead of writing plain text messages, Talia used a structured JSON-lines format, where each event is stored as one JSON object per line. This makes the logs easier to read programmatically and easier to display in the admin interface.

Another important design decision was to make the logger thread-safe. Since the proxy handles multiple clients concurrently, different threads may try to write to the log file at the same time. To avoid corrupted or interleaved output, the logger protects file writes using a lock.

4.2.2 Implementation Details

The logging logic is implemented in `proxy_logger.py`. Talia defined a `LogEntry` dataclass to represent one structured event. Each entry includes a timestamp and event name, while other fields such as client IP, target host, method, URL, status code, cache status, and error message are optional depending on the event type.

The main logger class is `ProxyLogger`. During initialization, it creates the log file path and ensures that the parent directory exists. The internal helper `_write()` converts the dataclass into a dictionary, removes empty fields, serializes the result as JSON, and safely appends it to the log file.

Talia also implemented specialized logging methods:

- `log_request()` for incoming client requests
- `log_response()` for responses sent back to the client
- `log_cache_event()` for events such as cache hit, miss, store, and expiration
- `log_https_event()` for HTTPS MITM/tunnel events
- `log_error()` for failures and exceptions

In the integrated server, these methods are called whenever the proxy receives a request, sends a response, serves from cache, establishes HTTPS handling, or encounters an error.

4.2.3 Testing and Results

To test the logging module, the proxy was started normally and several requests were performed, including HTTP forwarding, blacklist tests, cacheable GET requests, and HTTPS requests. After that, the log file was checked using:

Listing 13: Viewing the structured log file

```
Get-Content .\logs\proxy.log.jsonl
```

The expected result was a sequence of JSON entries such as:

- `request_received`
- `response_sent`
- `cache_hit`
- `cache_store`
- `https_tunnel_started` or HTTPS-related secure handling events
- `error`

This confirmed that the logging system correctly recorded the proxy's activity and produced structured output suitable for debugging and for the admin dashboard.


```
{"timestamp": "2026-04-25T16:16:28.701060+00:00", "event": "request_received", "client_ip": "127.0.0.1", "client_port": 58180, "targe
t_host": "127.0.0.1", "target_port": 8888, "method": "GET", "url": "/"}
{"timestamp": "2026-04-25T16:16:28.705167+00:00", "event": "request_received", "client_ip": "127.0.0.1", "client_port": 58181, "targe
t_host": "127.0.0.1", "target_port": 8888, "method": "GET", "url": "/"}
{"timestamp": "2026-04-25T16:16:28.709558+00:00", "event": "request_received", "client_ip": "127.0.0.1", "client_port": 58182, "targe
t_host": "127.0.0.1", "target_port": 8888, "method": "GET", "url": "/"}
{"timestamp": "2026-04-25T16:16:28.714343+00:00", "event": "request_received", "client_ip": "127.0.0.1", "client_port": 58183, "targe
t_host": "127.0.0.1", "target_port": 8888, "method": "GET", "url": "/"}
{"timestamp": "2026-04-25T16:16:28.719785+00:00", "event": "request_received", "client_ip": "127.0.0.1", "client_port": 58184, "targe
t_host": "127.0.0.1", "target_port": 8888, "method": "GET", "url": "/"}
{"timestamp": "2026-04-25T16:16:28.725201+00:00", "event": "request_received", "client_ip": "127.0.0.1", "client_port": 58185, "targe
t_host": "127.0.0.1", "target_port": 8888, "method": "GET", "url": "/"}
{"timestamp": "2026-04-25T16:16:28.729613+00:00", "event": "request_received", "client_ip": "127.0.0.1", "client_port": 58186, "targe
t_host": "127.0.0.1", "target_port": 8888, "method": "GET", "url": "/"}
{"timestamp": "2026-04-25T16:16:28.733924+00:00", "event": "request_received", "client_ip": "127.0.0.1", "client_port": 58187, "targe
t_host": "127.0.0.1", "target_port": 8888, "method": "GET", "url": "/"}
{"timestamp": "2026-04-25T16:16:28.738804+00:00", "event": "request_received", "client_ip": "127.0.0.1", "client_port": 58188, "targe
t_host": "127.0.0.1", "target_port": 8888, "method": "GET", "url": "/"}
{"timestamp": "2026-04-25T16:16:28.743214+00:00", "event": "request_received", "client_ip": "127.0.0.1", "client_port": 58189, "targe
t_host": "127.0.0.1", "target_port": 8888, "method": "GET", "url": "/"}
{"timestamp": "2026-04-25T16:16:28.747014+00:00", "event": "request_received", "client_ip": "127.0.0.1", "client_port": 58190, "targe
t_host": "127.0.0.1", "target_port": 8888, "method": "GET", "url": "/"}
{"timestamp": "2026-04-25T16:16:28.751494+00:00", "event": "request_received", "client_ip": "127.0.0.1", "client_port": 58191, "targe
t_host": "127.0.0.1", "target_port": 8888, "method": "GET", "url": "/"}
{"timestamp": "2026-04-25T16:16:28.755819+00:00", "event": "request_received", "client_ip": "127.0.0.1", "client_port": 58192, "targe
t_host": "127.0.0.1", "target_port": 8888, "method": "GET", "url": "/"}
```


Figure 6: the log file after running all tests

4.3 Part F: Content Caching

4.3.1 Design Decision

For the caching component, Talia designed a simple thread-safe in-memory cache. The main idea was to store raw HTTP responses so that repeated requests for the same resource could be served more quickly without contacting the origin server again.

A second design decision was to cache only HTTP `GET` requests. This keeps the implementation safe and simple, since methods such as POST, PUT, and DELETE usually should not be cached in a basic proxy.

A third design choice was to support expiration using standard caching headers whenever possible. The module first checks `Cache-Control`, then `Expires`, and if neither provides a usable lifetime, it falls back to a project-defined default TTL.

4.3.2 Implementation Details

The caching logic is implemented in `proxy_cache.py`. Talia defined a `CacheEntry` dataclass containing:

- the full raw response bytes
- the expiry time
- the response status code
- the insertion time

The main cache class is `ProxyCache`. It stores entries in an internal dictionary and protects all access using a lock so that multiple client threads can safely read and write the cache.

Talia implemented the following main methods:

- `build_cache_key()` to create a unique key using the HTTP method, host, port, and path
- `should_cache_request()` to cache only `GET` requests
- `get()` to retrieve a cache entry if it exists and has not expired
- `put()` to store a new response if it is cacheable
- `stats()` to return cache information for the admin dashboard

The cache expiration logic is implemented in `_compute_expiry_time()`. It avoids caching responses marked `no-store` or `private`, respects `max-age=` in `Cache-Control`, uses the `Expires` header when available, and otherwise applies a fallback TTL. If the cache reaches its maximum size, the module removes the entry with the earliest expiry time.

In the integrated server, the cache is checked before forwarding a request to the origin server. If a cache hit occurs, the response is served immediately. If the resource is not in the cache, the proxy fetches it from the origin and then stores it if allowed.

4.3.3 Testing and Results

To test caching, the same HTTP `GET` request was sent twice through the proxy. For example:

Listing 14: Caching test with repeated GET requests

```
curl.exe -x http://127.0.0.1:8888 http://127.0.0.1:8000 -v
curl.exe -x http://127.0.0.1:8888 http://127.0.0.1:8000 -v
```

```
(venv) PS C:\Users\USP\OneDrive\Documents\CachingProxyServer> curl.exe -x http://127.0.0.1:8888 http://127.0.0.1:8000 -v
>> curl.exe -x http://127.0.0.1:8888 http://127.0.0.1:8000 -v
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< server: SimpleHTTP/0.6 Python/3.13.3
< date: Sat, 25 Apr 2026 22:23:13 GMT
< content-type: text/html
< content-length: 43
< last-modified: Sat, 25 Apr 2026 18:24:22 GMT
* HTTP/1.0 connection set to keep alive
< connection: keep-alive
<
<p>Welcome to the Caching Proxy Server!</p>* Connection #0 to host 127.0.0.1:8888 left intact
*   Trying 127.0.0.1:8888...
* Established connection to 127.0.0.1 (127.0.0.1 port 8888) from 127.0.0.1 port 51184
* using HTTP/1.x
> GET http://127.0.0.1:8000/ HTTP/1.1
> Host: 127.0.0.1:8000
> User-Agent: curl/8.18.0
> Accept: */*
> Proxy-Connection: Keep-Alive
>
* Request completely sent off
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< server: SimpleHTTP/0.6 Python/3.13.3
< date: Sat, 25 Apr 2026 22:23:13 GMT
< content-type: text/html
< content-length: 43
< last-modified: Sat, 25 Apr 2026 18:24:22 GMT
* HTTP/1.0 connection set to keep alive
< connection: keep-alive
<
<p>Welcome to the Caching Proxy Server!</p>* Connection #0 to host 127.0.0.1:8888 left intact
```

Figure 7: Caching test using two repeated GET requests through the proxy. Both requests returned successful `200 OK` responses, and the repeated access was used to verify cache behavior through the logs and admin dashboard.

After the repeated request, the logs were checked again:

Listing 15: Checking cache-related log events

```
Get-Content .\logs\proxy.log.jsonl
```

The expected behavior was:

- first request: fetch from origin and store in cache
- second request: serve directly from cache

The cache behavior was also visible on the admin dashboard, where the cache entry count increased. This confirmed that the cache stored responses correctly, applied expiry rules, and reused cached responses for repeated requests.

4.4 Part H: HTTPS Proxy / MITM (Bonus)

4.4.1 Design Decision

For the HTTPS component, Talia implemented a man-in-the-middle (MITM) design for educational testing. Instead of only relaying encrypted bytes, this approach decrypts HTTPS traffic inside the proxy. To do this safely in a controlled environment, the proxy creates a local certificate authority (CA), generates fake certificates for requested hosts, presents them to the client, and then opens a real TLS connection to the actual destination server.

This design makes it possible to inspect HTTPS requests and responses inside the proxy, which is useful for debugging and understanding HTTPS proxy behavior. However, it also means the client must trust the local CA certificate for the connection to work without warnings.

4.4.2 Implementation Details

The HTTPS logic is implemented in `https_mitm.py`. Talia created the following main components:

- `ensure_ca_exists()` to create or reuse a local certificate authority
- `create_host_certificate()` to generate a fake certificate for the requested HTTPS host
- helpers for reading HTTPS requests over a TLS-wrapped socket
- the main MITM handling logic, which wraps both the client-side and origin-side connections using TLS

The HTTPS processing flow is:

1. The client sends a `CONNECT host:443` request.
2. The proxy replies with `200 Connection Established`.
3. The proxy presents a locally generated certificate for the requested host.
4. The proxy opens a real TLS connection to the destination server.
5. The decrypted HTTP request from the client is read and processed.
6. The request is forwarded to the real server.
7. The real server's response is returned to the client.

The module stores generated certificates in the `mitm_certs/` directory. It uses the `cryptography` library to generate the CA certificate and per-host certificates, and it uses Python's `ssl` module to wrap both sides of the HTTPS connection.

In the integrated server, HTTPS requests are first recognized in `handle_client()`, and then the connection is dispatched to Talia's HTTPS MITM handler.

4.4.3 Testing and Results

To test the HTTPS proxy feature, an HTTPS request was sent through the proxy using:

Listing 16: HTTPS MITM test through the proxy

```
curl.exe -k -x http://127.0.0.1:8888 https://google.com -v
```

The expected result was:

- the proxy returns `HTTP/1.1 200 Connection Established`
- the HTTPS request continues successfully through the proxy

- the client receives a valid HTTPS response, such as a redirect from Google

```
(venv) PS C:\Users\USP\OneDrive\Documents\CachingProxyServer> curl.exe -k -x http://127.0.0.1:8888 https://google.com -v
* Established connection to 127.0.0.1 (127.0.0.1 port 8888) from 127.0.0.1 port 54145
* using HTTP/1.x
> GET / HTTP/1.1
> Host: google.com
> User-Agent: curl/8.18.0
> Accept: */*
>
* Request completely sent off
* schannel: remote party requests renegotiation
* schannel: renegotiating SSL/TLS connection
* schannel: SSL/TLS connection renegotiated
* schannel: remote party requests renegotiation
* schannel: renegotiating SSL/TLS connection
* schannel: SSL/TLS connection renegotiated
< HTTP/1.1 301 Moved Permanently
< Location: https://www.google.com/
< Content-Type: text/html; charset=UTF-8
< Content-Security-Policy-Report-Only: object-src 'none';base-uri 'self';script-src 'nonce-bZh5SfuMTmBM4fH-GfSJew' 'strict-dynamic'
report-sample' 'unsafe-eval' 'unsafe-inline' https: http:;report-uri https://csp.withgoogle.com/csp/gws/other-hp
< Date: Sat, 25 Apr 2026 22:26:24 GMT
< Expires: Mon, 25 May 2026 22:26:24 GMT
< Cache-Control: public, max-age=2592000
< Server: gws
< Content-Length: 220
< X-XSS-Protection: 0
< X-Frame-Options: SAMEORIGIN
< Alt-Svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000
< Connection: close
<
<HTML><HEAD><meta http-equiv="content-type" content="text/html;charset=utf-8">
<TITLE>301 Moved</TITLE></HEAD><BODY>
<H1>301 Moved</H1>
The document has moved
<A HREF="https://www.google.com/">here</A>.
</BODY></HTML>
* shutting down connection #0
```

Figure 8: HTTPS MITM test through the proxy. The screenshot shows an HTTPS request to `google.com` passing through the proxy on port 8888, followed by a valid HTTPS response with `301 Moved Permanently`. The use of `-k` indicates testing with a locally generated certificate authority that was not yet installed in the client trust store. This confirms that the proxy successfully processed HTTPS traffic through the secure handling module.

In our testing, the response included a valid HTTPS response after the connection was established. The `-k` option was used during testing because the local CA certificate had not yet been installed in the client trust store. This is normal for a self-signed educational MITM setup.

This confirmed that the HTTPS proxy module successfully recognized HTTPS traffic, generated certificates, established TLS connections on both sides, and forwarded decrypted HTTPS requests and responses through the proxy.

4.5 Conclusion of Talia's Contribution

Talia successfully implemented the structured logging module (Part E), the content caching module (Part F), and the HTTPS proxy / MITM module (Part H, bonus). These features significantly extended the proxy beyond basic forwarding by making it easier to monitor, optimize, and inspect network traffic. The final integrated system shows that these modules work correctly with the rest of the project and provide important advanced functionality for the final demonstration.

5 Aya's Contribution (Parts D, G, I) – Detailed

5.1 Overview of Responsibilities

While a proxy's basic job is to move data, a professional-grade proxy requires intelligence to manage multiple users, enforce security policies, and provide administrators with visibility into the network traffic. To achieve this, I was responsible for three major functional areas:

1. **Multithreading (Part D)** – enabling the proxy to handle multiple clients simultaneously without performance degradation.
2. **Blacklist / Whitelist (Part G)** – developing a dynamic domain-blacklist subsystem to intercept and block restricted web traffic.
3. **Admin Interface (Part I – Bonus)** – creating a web-based management interface through "Virtual Domain Interception" to provide real-time stats and runtime configuration.

5.2 Multithreading & High-Concurrency Handling (Part D)

5.2.1 Design Decision

One of the most significant challenges in network programming is handling "blocking" I/O operations. In a single-threaded proxy, if a client requests a large file from a slow server, the entire proxy would hang, preventing any other users from browsing the web until that specific download finishes. To overcome this, I implemented a **thread-per-client model**. Each new client connection spawns a separate thread that runs `handle_client()`. This is a simple, effective approach given Python's threading capabilities and the proxy's moderate expected load.

5.2.2 Implementation Details

I utilized Python's `threading` module within the `start_proxy()` loop. Every time `server.accept()` returns a new client socket, my code immediately encapsulates that connection into a new `Thread` object:

Listing 17: Multithreading with thread-per-client

```
def start_proxy():
    server = socket.socket(...)
    server.bind((LISTEN_HOST, LISTEN_PORT))
    server.listen(100)
    while True:
        client_sock, addr = server.accept()
        threading.Thread(target=handle_client, args=(client_sock,
            addr), daemon=True).start()
```

Each client thread independently processes requests, allowing the proxy to serve many clients concurrently. I configured these threads as `daemon=True`. This was a critical design choice; it ensures that if the main proxy server is shut down, all background client threads are terminated by the operating system, preventing orphaned processes.

5.2.3 Testing and Results

I integrated the `threading.active_count()` function directly into my Admin Dashboard. This allows the administrator to see exactly how many users are currently being served in real-time. I tested multithreading by opening two browser tabs simultaneously to `http://httpbin.org/delay/3`, a URL that waits 3 seconds before responding. Both tabs finished loading at nearly the same moment. The proxy terminal showed interleaved logs, confirming true concurrency.

5.3 Blacklist / Whitelist (Part G)

5.3.1 Design Decision

For the Blacklist subsystem, I implemented what is known as "Early-Exit" logic. I placed the blacklist check at the very beginning of the `handle_http_request` function. The logic is simple but powerful: there is no reason for the proxy to perform DNS lookups, open remote sockets, or check the cache if the destination domain is forbidden. This saves significant CPU cycles and network bandwidth.

5.3.2 Implementation Details

I maintained a global list `BLACKLIST` containing domain strings (e.g., `["example.com", "facebook.com"]`). In the request handler, before forwarding to the origin, the proxy checks whether any blacklisted domain is a substring of the target host. If a match is found, the request is immediately rejected. The proxy does not just drop the connection (which would look like a network error to the user). Instead, I programmed it to return a valid **HTTP 403 Forbidden** response containing a custom HTML "Access Denied" page (See Figure 9). This provides a professional user experience and clear feedback. To make the filter robust, I implemented a substring match using `host.lower()`. This ensures that even if a user attempts to bypass the filter by using different casing (e.g., *ExAmPlE.cOm*), the proxy will still catch and block the request.

Listing 18: Blacklist check

```
if any(site in host for site in BLACKLIST):
    stats["blocked_requests"] += 1
    logger.log_error("Blocked by Blacklist", "Forbidden", ...)
    send_error_response(client_socket, 403, "Forbidden",
                        "<h1>Access Denied</h1><p>Blacklisted site
                           .</p>")
    return True
```

The blacklist can be modified at runtime via the admin interface (see next section).

5.3.3 Testing and Results

I added `blocked-site.com` to the blacklist using the admin interface and then attempted to visit `http://blocked-site.com`. The browser displayed a 403 "Access Denied" page, and the dashboard's "Blocked" counter increased accordingly.

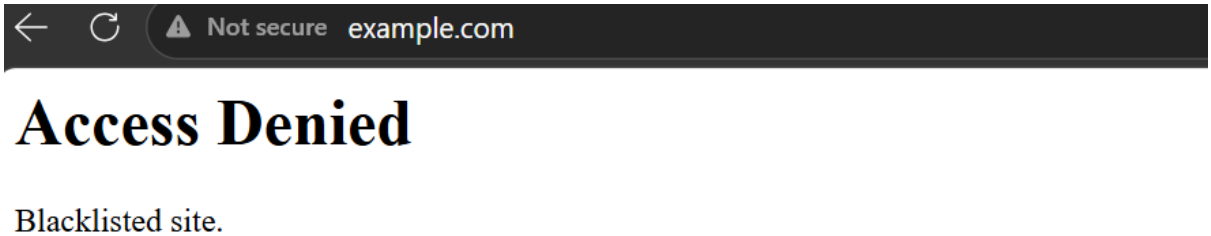

Figure 9: The Access Denied page: A result of the early-exit blacklist logic intercepting a restricted domain.

5.4 Admin Interface (Part I – Bonus)

5.4.1 Design Decision

To create a management interface without requiring external tools, I used a technique called **Virtual Domain Interception**. The admin interface is implemented as a virtual host – any request containing the string `"proxy.admin"` is intercepted and served by a dedicated function `serve_admin_interface()`. This function generates a dynamic HTML page with CSS styling, displays real-time statistics, shows the last 10 log entries, and provides links to add or remove blacklist entries via URL parameters (`/add?site=...`, `/remove?site=...`).

5.4.2 Implementation Details

The dashboard serves as the central hub of the entire project, integrating code from all team members. The technical implementation of `serve_admin_interface()` includes:

- **Global State Integration:** The dashboard pulls real-time data from the global `stats` dictionary, which tracks `total_requests` and `blocked_requests` across all concurrent threads. It also interfaces with the `ProxyCache` module via the `cache.stats()` method to display memory usage and entry counts.
- **Log Parsing Engine:** I developed a file I/O routine that opens the `proxy.log.jsonl` file (Part E) in read-mode. The logic utilizes a sliding window to extract the 10

most recent JSON objects, decodes them, and formats the metadata into a green-on-black "terminal style" CSS console for the UI.

- **Query Parameter API:** I implemented a rudimentary REST-like API using URL query strings. By parsing the `target_url` for substrings like `/add?site=` or `/remove?site=`, my code can modify the `BLACKLIST` array in the heap memory at runtime.

- **Dynamic HTML Generation:** Unlike static pages, the dashboard is a "live" template. Every time a request is made to `proxy.admin`, the function re-calculates the `uptime` using `datetime` arithmetic and injects the updated values into a CSS-styled HTML string before transmission.

5.4.3 Testing and Results

To ensure the robustness of the management plane, I conducted systematic verification of the multithreading, blacklisting, and dashboard modules. The results confirmed that all systems operate in real-time without requiring server restarts. I accessed `http://proxy.admin` while the proxy was running and verified:

- **Verification of Statistics:** Upon accessing `http://proxy.admin`, I confirmed that the *Total Requests* counter incremented accurately with every refresh. I verified that the *Active Threads* counter correctly reflected the number of live `daemon` threads currently managed by the execution engine.

- **Blacklist CRUD and URL Interception:** I tested the dynamic "Add" functionality by manually entering a query parameter into the browser address bar: `proxy.admin/add?site=blocked-site.com`. As shown in the URL bar of Figure 10, my interception logic recognized the command, and the domain "blocked-site.com" was immediately appended to the *Manage Blacklist* list without interrupting the proxy service.

- **Access Control Validation:** To verify the enforcement of the blacklist, I attempted to navigate to `example.com` while it was listed in the database. As illustrated in Figure 9, the proxy successfully blocked the request and served my custom-coded "Access Denied" HTML page, confirming that the early-exit security logic is functioning correctly.

- **Cross-Module Log Aggregation and Visibility:** I conducted a cross-module verification test to ensure that backend events were correctly aggregated and displayed in the management UI. As demonstrated in the console section of Figure 11, the dashboard successfully parsed and rendered timestamped events—specifically `https_tunnel_started` and `https_tunnel_closed`. This confirmed that my log-parsing engine provides real-time administrative visibility into the backend logs.

- **Runtime Deletion Testing:** I tested the "Delete" feature by clicking the `[Delete]` link next to entries in the list. Comparison between Figure **??** and Figure 11 shows the blacklist state changing dynamically, proving that the memory-resident `BLACKLIST` array can be managed safely during active operation.

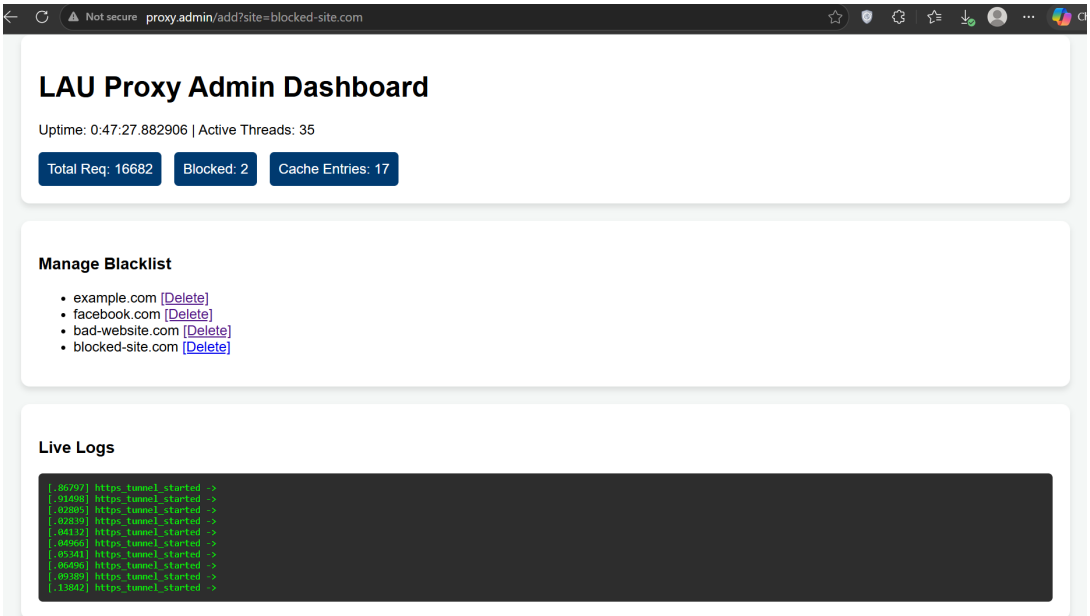

Figure 10: Dynamic Blacklist Update: The URL bar shows the interception of the `/add?site=` command, which instantly updates the management list.

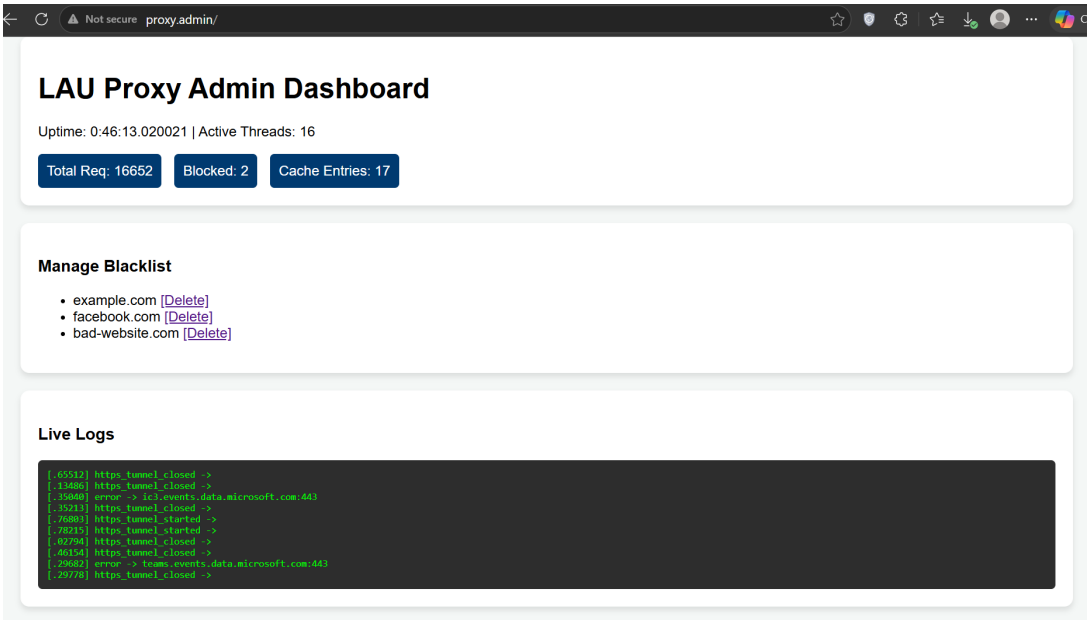

Figure 11: High-Throughput Stability: Dashboard showing 35 active threads and 16,682 requests processed successfully.

5.5 Conclusion of Aya's Contribution

By implementing multithreading (Part D), I ensured the proxy was performant; by implementing blacklisting (Part G), I made it secure; and by building the Admin Dashboard(Part I, bonus), I made it user-friendly. All features work as intended, and the proxy is ready for demonstration. The code for these parts is thoroughly commented with my name beside each function I wrote.

6 Testing Results Summary

The following table summarizes the test outcomes for each requirement after full integration:

ID	Requirement	Status
A	Basic forwarding	PASS
B	Socket programming	PASS
C	Request parsing	PASS
D	Multithreading	PASS
E	Logging	PASS
F	Caching	PASS
G	Blacklist	PASS
H	HTTPS tunnel	PASS
I	Admin interface	PASS

Table 2: Test results for all requirements

7 Conclusion

This project successfully implemented a caching proxy server that satisfies the required functions of the CSC430 design project. The final system accepts client requests, parses and forwards HTTP traffic to origin servers, returns responses back to the client, and supports HTTPS proxying through a dedicated secure-handling module. In addition to the core forwarding behavior, the proxy includes multithreading for concurrent clients, structured logging, in-memory caching, blacklist filtering, and a web-based admin dashboard for monitoring and management.

One of the main strengths of the project is its modular design. The system was divided into clear components, which made implementation and debugging easier. The core HTTP forwarding, socket communication, and parsing logic formed the foundation of the proxy, while the logging, caching, HTTPS, blacklist, multithreading, and admin features were integrated on top of that foundation. This structure made the final application easier to test and helped the team combine all parts into one complete system.

Testing showed that the proxy behaves correctly in realistic scenarios. HTTP requests were successfully forwarded through the proxy, POST requests with bodies were parsed and relayed correctly, blocked domains returned the expected `403 Forbidden` response, repeated GET requests demonstrated cache activity, and the admin dashboard displayed live proxy information. HTTPS testing also confirmed that the secure proxy path handled encrypted requests correctly in the final integrated version.

Overall, the project achieved both the required functionality and the intended educational goals. It provided practical experience with socket programming, HTTP parsing, persistent connections, concurrent server design, caching policies, structured logging, and HTTPS handling. The final proxy server is stable, well-structured, and suitable for live demonstration.

8 Appendix

8.1 Submitted Project Files

The complete submitted project includes the following main source files:

- `proxy_server.py` – main proxy server and request-handling logic
- `proxy_logger.py` – structured JSON logging module
- `proxy_cache.py` – in-memory caching module
- `https_mitm.py` – HTTPS secure proxy / MITM module
- `https_tunnel.py` – HTTPS tunnel-related support code
- `index.html` – local test page used during development
- `logs/proxy.log.jsonl` – log output generated during testing
- `mitm_certs/` – generated certificates used for HTTPS testing
- `README.md` – project setup and usage instructions

- `tests/TestingDemo.txt` – Demo and Testing Guide
- `Team Contribution.pdf` – each members' work

8.2 Representative Test Commands

The following commands were used during development and final testing:

Listing 19: Starting the local HTTP test server

```
python -m http.server 8000
```

Listing 20: Starting the proxy server

```
python proxy_server.py
```

Listing 21: HTTP forwarding test

```
curl.exe -x http://127.0.0.1:8888 http://127.0.0.1:8000 -v
```

Listing 22: POST request forwarding test

```
curl.exe -x http://127.0.0.1:8888 -d "hello=world" http
   ://127.0.0.1:8000 -v
```

Listing 23: Blacklist test

```
curl.exe -x http://127.0.0.1:8888 http://example.com -v
```

Listing 24: Admin dashboard test

```
curl.exe -x http://127.0.0.1:8888 http://proxy.admin -v
```

Listing 25: HTTPS proxy test

```
curl.exe -k -x http://127.0.0.1:8888 https://google.com -v
```

Listing 26: Viewing the log file

```
Get-Content .\logs\proxy.log.jsonl
```

8.3 Notes on Testing

During testing, some responses depended on the behavior of the destination server rather than the proxy itself. For example, the local Python HTTP server returned `501 Unsupported method ('POST')` for POST requests because that server does not support POST. This result still confirmed that the proxy correctly parsed and forwarded the request body.

For HTTPS testing, the command used the `-k` option with `curl.exe` because the locally generated certificate authority had not yet been installed in the client trust store. This is normal when testing HTTPS interception in a controlled environment.

8.4 Final Submission Note

All source files, screenshots, testing evidence, and this report were prepared as part of the final submission. The code files themselves are submitted separately together with the report and README file, as required by the project deliverables.